Entrega 3 - Projeto do Algoritmo

Sistema de Controle de Estoque

Alunos:

- Eduardo Santos do Nascimento
- Agnaldo Júnior
- Patrick Silva

1. Estratégia utilizada

1.1 Visão geral

O problema foi dividido em módulos pequenos, cada um com uma responsabilidade, seguindo a modularização da Entrega 2:

Módulo	O que faz	Funções
Programa principal	Mostra o menu e chama o módulo escolhido	`ControleDeEstoque`
Módulo 1: Cadastros	Cadastra um produto novo com validações	`cadastrar_produto`
Módulo 2: Movimentações	Registra entradas e saídas de estoque	`buscar_produto`, `registrar_entrada`, `registrar_saida`
Módulo 3: Consultas	Mostra os produtos na tela	`listar_produtos`, `consultar_produto`, `exibir_produto`
Módulo 4: Relatórios	Mostra só os produtos que precisam de reposição	`relatorio_estoque_baixo`
Módulo 5: CSV	Lê e grava o arquivo de dados	`carregar_produtos`, `salvar_produtos`

O fluxo de uso do sistema é sempre o mesmo:

1. Ao abrir, o programa carrega os produtos do CSV para o vetor `produtos`.
2. O menu é exibido e o usuário escolhe uma opção.
3. O programa chama a função do módulo correspondente.
4. Se a opção alterou o estoque (cadastro, entrada ou saída) e deu certo, o CSV é salvo.
5. O menu volta a aparecer até o usuário escolher "Sair".

Os módulos 3 e 4 só leem o vetor `produtos` e nunca alteram nada, por isso o menu não precisa salvar o CSV depois deles. A função `exibir_produto` é usada pela listagem, pela consulta, pelo relatório e pelas movimentações, então a forma de mostrar um produto e o alerta "Estoque Baixo / Necessidade de Reposição" ficam escritos em um lugar só.

1.2 Estrutura de dados e persistência

Os produtos ficam em um **vetor de registros** (`produtos`), mantido em memória durante toda a execução. Cada elemento é um registro do tipo `Produto`, com os campos da Entrega 2. O vetor foi escolhido porque as operações são simples e o volume de dados é pequeno: percorrer a lista para listar, gerar o relatório ou localizar um código é suficiente.

O **arquivo CSV** guarda os dados entre uma execução e outra. Ele não é lido a cada operação:

1. `carregar_produtos()` é chamada uma única vez, na abertura do programa;
2. todas as consultas e validações trabalham apenas sobre o vetor;
3. `salvar_produtos(produtos)` regrava o arquivo inteiro depois de cada alteração concluída com sucesso.

Assim o arquivo só é regravado quando a operação terminou válida. Uma saída bloqueada por saldo insuficiente, por exemplo, não altera nada.

Formato do arquivo (cabeçalho, separador `;`, uma linha por produto):

```
codigo_produto;nome_produto;categoria_produto;quantidade_atual;quantidade_minima
101;Teclado Mecanico;Perifericos;25;10
```

1.3 Regras de negócio e validações

A ideia principal é **validar tudo antes de alterar qualquer coisa**. Cada função lê os dados, faz as verificações e só mexe no vetor depois que todas passaram. Se uma verificação falhar, a função mostra a mensagem e retorna `falso`, e o menu não salva o CSV.

Verificação	Onde	Mensagem
Código repetido, nome ou categoria vazios, quantidades negativas	Cadastro	Mensagem específica de cada campo
Produto não existe (`buscar_produto` retorna `-1`)	Entrada e saída	"Produto não encontrado."
Quantidade menor ou igual a zero	Entrada e saída	"A quantidade deve ser maior que zero."
`quantidade_saida > quantidade_atual`	Só saída	"Quantidade insuficiente para esta saída"

Com essas regras:

- **O estoque nunca fica negativo.** O cadastro só aceita quantidade inicial `>= 0`, a entrada só soma valores positivos e a saída só subtrai quando `quantidade_saida <= quantidade_atual`. O menor saldo possível é `0`.
- **Entrada e saída precisam ser maiores que zero.** Uma "entrada de -50" seria uma saída sem passar pela validação de saldo, e o valor `0` não muda nada no estoque.

- **Só quantidades inteiras.** As quantidades são declaradas como `inteiro`, como pede a restrição de tipagem da Entrega 2.
- **O código é único.** O cadastro não aceita código repetido, então `buscar_produto` sempre encontra o produto certo.
- **O usuário vê o resultado na hora.** Depois de uma entrada ou saída aceita, o produto é exibido com `exibir_produto`. Se ficou abaixo do mínimo, o alerta de estoque baixo já aparece.

2. Pseudocódigo

2.1 Tipo de dado e constantes

```
TIPO Produto
    codigo_produto: inteiro
    nome_produto: cadeia
    categoria_produto: cadeia
    quantidade_atual: inteiro
    quantidade_minima: inteiro
FIMTIPO

CONSTANTE
    MAX_PRODUTOS = 100
    ARQUIVO_CSV = "produtos.csv"
```

2.2 Programa principal

```
ALGORITMO ControleDeEstoque

VAR
    produtos: vetor[1..MAX_PRODUTOS] de Produto
    opcao: inteiro

INICIO
    produtos <- carregar_produtos()

    REPITA
        ESCREVA "===== CONTROLE DE ESTOQUE ====="
        ESCREVA "1 - Cadastrar produto"
        ESCREVA "2 - Registrar entrada"
        ESCREVA "3 - Registrar saida"
        ESCREVA "4 - Listar produtos"
        ESCREVA "5 - Consultar produto"
        ESCREVA "6 - Relatorio de estoque baixo"
        ESCREVA "7 - Sair"
        ESCREVA "Escolha uma opcao: "
        LEIA opcao

        ESCOLHA opcao
            CASO 1
                SE cadastrar_produto(produtos) ENTAO
                    salvar_produtos(produtos)
                FIMSE
            CASO 2
                SE registrar_entrada(produtos) ENTAO
                    salvar_produtos(produtos)
                FIMSE
            CASO 3
                SE registrar_saida(produtos) ENTAO
                    salvar_produtos(produtos)
                FIMSE
            CASO 4
                listar_produtos(produtos)
            CASO 5
                consultar_produto(produtos)
            CASO 6
                relatorio_estoque_baixo(produtos)
            CASO 7
                ESCREVA "Encerrando o sistema..."
            SENAO
                ESCREVA "Opcao invalida. Tente novamente."
        FIMESCOLHA
    ATE opcao = 7
FIMALGORITMO
```

O menu só grava no CSV quando a operação retorna `verdadeiro`. Por isso `cadastrar_produto`, `registrar_entrada` e `registrar_saida` retornam `logico`.

2.3 Módulo 5: Leitura e escrita no CSV

```
FUNCAO carregar_produtos(): vetor de Produto
VAR
    produtos: vetor[1..MAX_PRODUTOS] de Produto
    linha: cadeia
    campos: vetor de cadeia
    total: inteiro

INICIO
    total <- 0

    SE NAO existe_arquivo(ARQUIVO_CSV) ENTAO
        RETORNE produtos
    FIMSE

    abrir_arquivo(ARQUIVO_CSV, leitura)

    SE NAO fim_arquivo(ARQUIVO_CSV) ENTAO
        LEIA_LINHA(ARQUIVO_CSV, linha)
    FIMSE

    ENQUANTO NAO fim_arquivo(ARQUIVO_CSV) E total < MAX_PRODUTOS FACA
        LEIA_LINHA(ARQUIVO_CSV, linha)
        SE linha <> "" ENTAO
            campos <- dividir(linha, ";")
            total <- total + 1
            produtos[total].codigo_produto    <- inteiro(campos[1])
            produtos[total].nome_produto      <- campos[2]
            produtos[total].categoria_produto <- campos[3]
            produtos[total].quantidade_atual  <- inteiro(campos[4])
            produtos[total].quantidade_minima <- inteiro(campos[5])
        FIMSE
    FIMENQUANTO

    fechar_arquivo(ARQUIVO_CSV)
    RETORNE produtos
FIMFUNCAO
```

A primeira linha lida é o cabeçalho e é descartada. Se o arquivo ainda não existe, o programa começa com a lista vazia.

```
PROCEDIMENTO salvar_produtos(produtos)
VAR
    i: inteiro

INICIO
    abrir_arquivo(ARQUIVO_CSV, escrita)
    ESCREVA_LINHA(ARQUIVO_CSV,
"codigo_produto;nome_produto;categoria_produto;quantidade_atual;quantidade_minima")

    PARA i DE 1 ATE tamanho(produtos) FACA
        ESCREVA_LINHA(ARQUIVO_CSV,
            produtos[i].codigo_produto + ";" +
            produtos[i].nome_produto + ";" +
            produtos[i].categoria_produto + ";" +
            produtos[i].quantidade_atual + ";" +
            produtos[i].quantidade_minima)
    FIMPARA

    fechar_arquivo(ARQUIVO_CSV)
FIMPROCEDIMENTO
```

2.4 Módulo 1: Gestão de Cadastros

```
FUNCAO cadastrar_produto(produtos): logico
VAR
    codigo_produto: inteiro
    nome_produto: cadeia
    categoria_produto: cadeia
    quantidade_atual: inteiro
    quantidade_minima: inteiro

INICIO
    SE tamanho(produtos) >= MAX_PRODUTOS ENTAO
        ESCREVA "Limite de produtos atingido."
        RETORNE falso
    FIMSE

    ESCREVA "Codigo do produto: "
    LEIA codigo_produto
    SE codigo_produto <= 0 ENTAO
        ESCREVA "Codigo invalido."
        RETORNE falso
    FIMSE

    SE buscar_produto(produtos, codigo_produto) <> -1 ENTAO
        ESCREVA "Ja existe um produto com este codigo."
        RETORNE falso
    FIMSE

    ESCREVA "Nome do produto: "
    LEIA nome_produto
    SE nome_produto = "" ENTAO
        ESCREVA "O nome do produto e obrigatorio."
        RETORNE falso
    FIMSE

    ESCREVA "Categoria do produto: "
    LEIA categoria_produto
    SE categoria_produto = "" ENTAO
        ESCREVA "A categoria do produto e obrigatoria."
        RETORNE falso
    FIMSE

    ESCREVA "Quantidade inicial: "
    LEIA quantidade_atual
    SE quantidade_atual < 0 ENTAO
        ESCREVA "A quantidade inicial nao pode ser negativa."
        RETORNE falso
    FIMSE

    ESCREVA "Quantidade minima: "
    LEIA quantidade_minima
    SE quantidade_minima < 0 ENTAO
        ESCREVA "A quantidade minima nao pode ser negativa."
        RETORNE falso
```

```
    FIMSE

    adicionar(produtos, Produto{codigo_produto, nome_produto, categoria_produto,
                                quantidade_atual, quantidade_minima})
    ESCREVA "Produto cadastrado com sucesso."
    RETORNE verdadeiro
FIMFUNCAO
```

As validações acontecem antes de qualquer gravação: se uma delas falhar, o vetor e o CSV continuam como estavam.

2.5 Módulo 2: Processamento de Movimentações

```
FUNCAO buscar_produto(produtos, codigo): inteiro
VAR
    i: inteiro

INICIO
    PARA i DE 1 ATE tamanho(produtos) FACA
        SE produtos[i].codigo_produto = codigo ENTAO
            RETORNE i
        FIMSE
    FIMPARA

    RETORNE -1
FIMFUNCAO
```

A busca é sequencial e para assim que encontra o código. Se o laço termina sem encontrar, retorna `-1`. Essa função também é usada pelo cadastro (para checar código repetido) e pela consulta por código.

```
FUNCAO registrar_entrada(produtos): logico
VAR
    codigo_produto: inteiro
    quantidade_entrada: inteiro
    posicao: inteiro

INICIO
    ESCREVA "Codigo do produto: "
    LEIA codigo_produto

    posicao <- buscar_produto(produtos, codigo_produto)
    SE posicao = -1 ENTAO
        ESCREVA "Produto não encontrado."
        RETORNE falso
    FIMSE

    ESCREVA "Quantidade de entrada: "
    LEIA quantidade_entrada
    SE quantidade_entrada <= 0 ENTAO
        ESCREVA "A quantidade deve ser maior que zero."
        RETORNE falso
    FIMSE

    produtos[posicao].quantidade_atual <- produtos[posicao].quantidade_atual +
quantidade_entrada

    ESCREVA "Entrada registrada com sucesso."
    exibir_produto(produtos[posicao])
    RETORNE verdadeiro
FIMFUNCAO
```

```
FUNCAO registrar_saida(produtos): logico
VAR
    codigo_produto: inteiro
    quantidade_saida: inteiro
    posicao: inteiro

INICIO
    ESCREVA "Codigo do produto: "
    LEIA codigo_produto

    posicao <- buscar_produto(produtos, codigo_produto)
    SE posicao = -1 ENTAO
        ESCREVA "Produto não encontrado."
        RETORNE falso
    FIMSE

    ESCREVA "Quantidade de saida: "
    LEIA quantidade_saida
    SE quantidade_saida <= 0 ENTAO
        ESCREVA "A quantidade deve ser maior que zero."
        RETORNE falso
    FIMSE

    SE quantidade_saida <= produtos[posicao].quantidade_atual ENTAO
        produtos[posicao].quantidade_atual <- produtos[posicao].quantidade_atual -
quantidade_saida
        ESCREVA "Saída registrada com sucesso."
        exibir_produto(produtos[posicao])
        RETORNE verdadeiro
    SENAO
        ESCREVA "Quantidade insuficiente para esta saída"
        RETORNE falso
    FIMSE
FIMFUNCAO
```

O último `SE ... SENAO` é a Condição de Validação de Saída da Entrega 2. Uma saída igual ao saldo é permitida e deixa o produto com `0`.

2.6 Módulo 3: Consultas e Exibição

```
PROCEDIMENTO exibir_produto(produto)
INICIO
    ESCREVA "Codigo: ", produto.codigo_produto
    ESCREVA "Nome: ", produto.nome_produto
    ESCREVA "Categoria: ", produto.categoria_produto
    ESCREVA "Quantidade atual: ", produto.quantidade_atual
    ESCREVA "Quantidade minima: ", produto.quantidade_minima

    SE produto.quantidade_atual < produto.quantidade_minima ENTAO
        ESCREVA "Status: Estoque Baixo / Necessidade de Reposição"
    SENAO
        ESCREVA "Status: OK"
    FIMSE

    ESCREVA "------------------------------"
FIMPROCEDIMENTO
```

```
PROCEDIMENTO listar_produtos(produtos)
VAR
    i: inteiro

INICIO
    SE tamanho(produtos) = 0 ENTAO
        ESCREVA "Nenhum produto cadastrado."
    SENAO
        ESCREVA "===== LISTA DE PRODUTOS ====="
        PARA i DE 1 ATE tamanho(produtos) FACA
            exibir_produto(produtos[i])
        FIMPARA
    FIMSE
FIMPROCEDIMENTO
```

```
PROCEDIMENTO consultar_produto(produtos)
VAR
    tipo_busca: inteiro
    codigo: inteiro
    nome: cadeia
    posicao: inteiro
    encontrou: logico
    i: inteiro

INICIO
    ESCREVA "Buscar por: 1 - Codigo  2 - Nome"
    LEIA tipo_busca

    SE tipo_busca = 1 ENTAO
        ESCREVA "Codigo do produto: "
        LEIA codigo
        posicao <- buscar_produto(produtos, codigo)
        SE posicao = -1 ENTAO
            ESCREVA "Produto não encontrado."
        SENAO
            exibir_produto(produtos[posicao])
        FIMSE
    SENAO
        SE tipo_busca = 2 ENTAO
            ESCREVA "Nome do produto: "
            LEIA nome
            encontrou <- falso
            PARA i DE 1 ATE tamanho(produtos) FACA
                SE produtos[i].nome_produto = nome ENTAO
                    exibir_produto(produtos[i])
                    encontrou <- verdadeiro
                FIMSE
            FIMPARA
            SE NAO encontrou ENTAO
                ESCREVA "Produto não encontrado."
            FIMSE
        SENAO
            ESCREVA "Opção inválida."
        FIMSE
    FIMSE
FIMPROCEDIMENTO
```

Na busca por nome a lista é percorrida inteira, porque o nome não é único e pode haver mais de um produto com o mesmo nome. Todos são exibidos.

2.7 Módulo 4: Monitoramento e Relatórios

```
PROCEDIMENTO relatorio_estoque_baixo(produtos)
VAR
    i: inteiro
    total_baixo: inteiro

INICIO
    ESCREVA "Relatório de Estoque Baixo"
    total_baixo <- 0

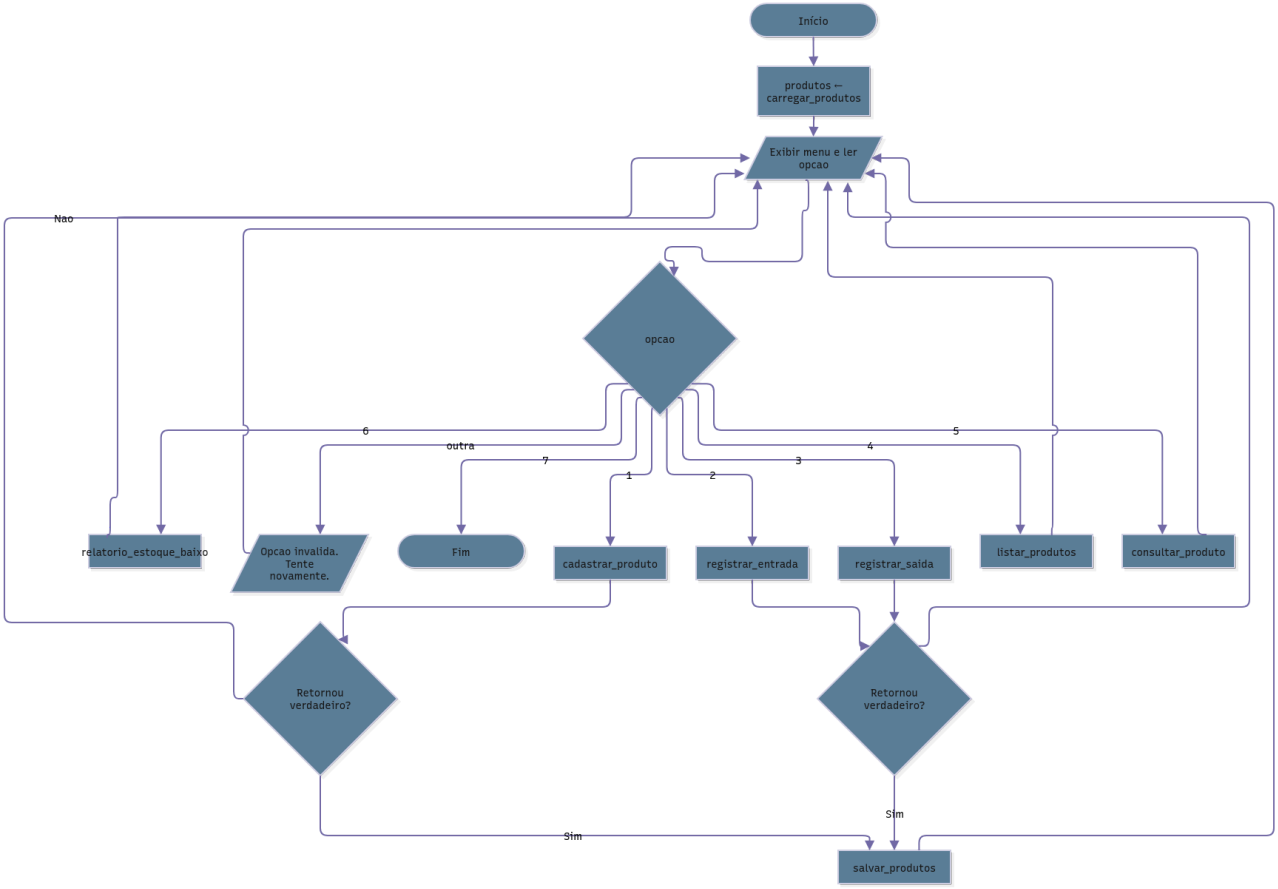
    PARA i DE 1 ATE tamanho(produtos) FACA
        SE produtos[i].quantidade_atual < produtos[i].quantidade_minima ENTAO
            exibir_produto(produtos[i])
            total_baixo <- total_baixo + 1
        FIMSE
    FIMPARA

    SE total_baixo = 0 ENTAO
        ESCREVA "Nenhum produto com estoque baixo."
    FIMSE
FIMPROCEDIMENTO
```

O relatório usa a Condição de Alerta de Estoque Baixo da Entrega 2 (`quantidade_atual < quantidade_minima`). O contador `total_baixo` serve para avisar quando nenhum produto precisa de reposição.

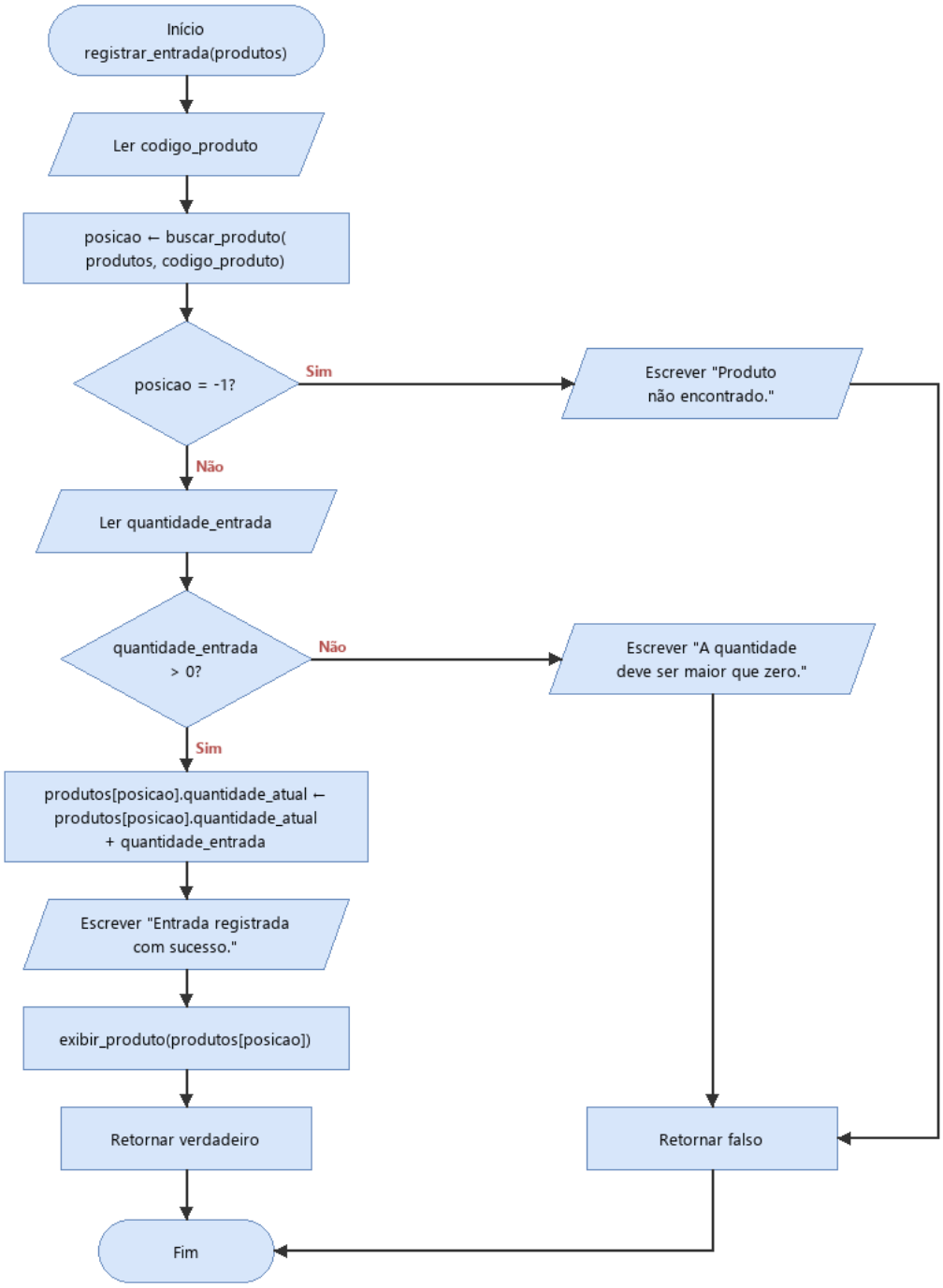
3. Fluxogramas

3.1 Programa principal (menu)



Fluxograma do programa principal

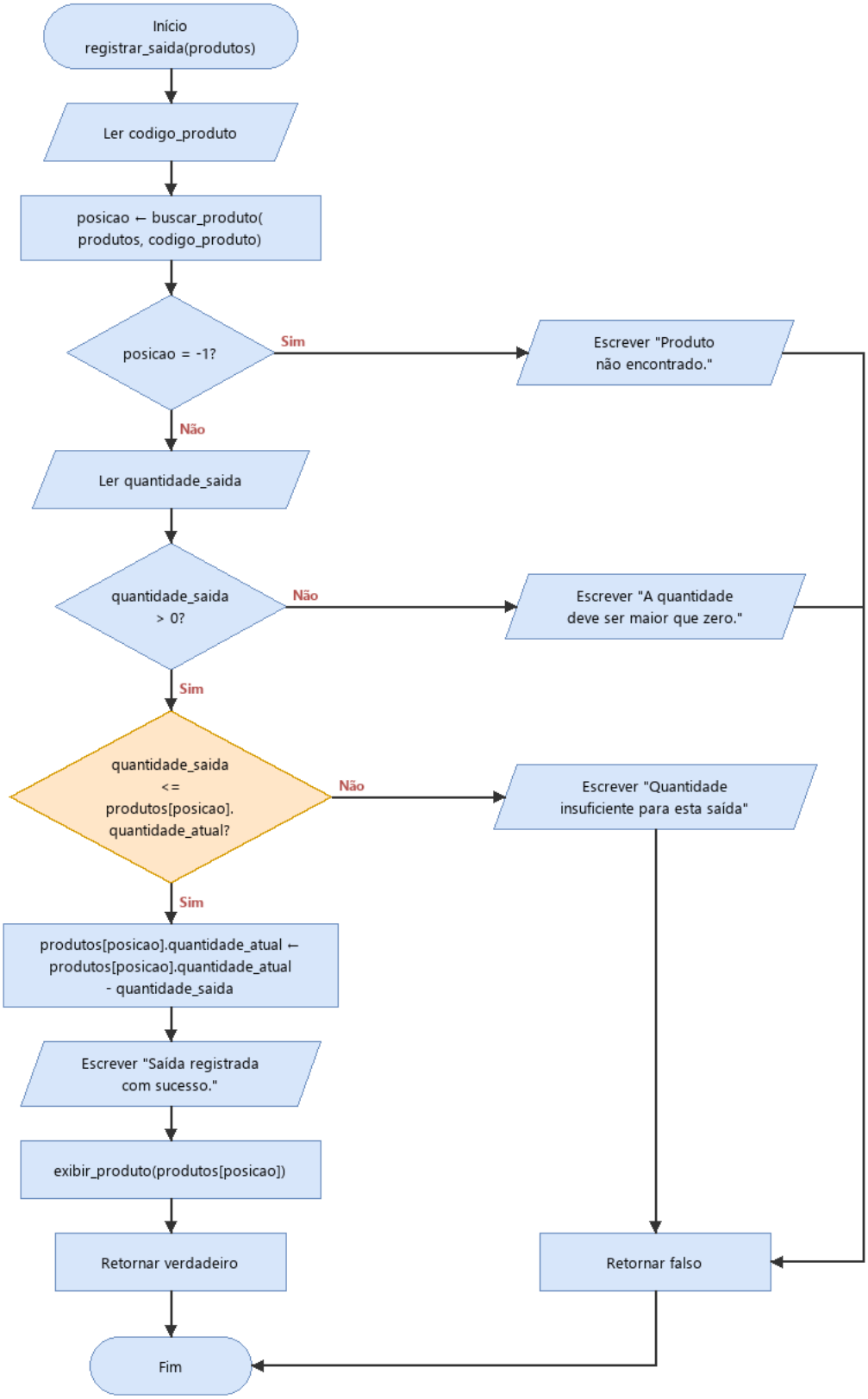
3.2 Entrada de produtos



Fluxograma da entrada de produtos

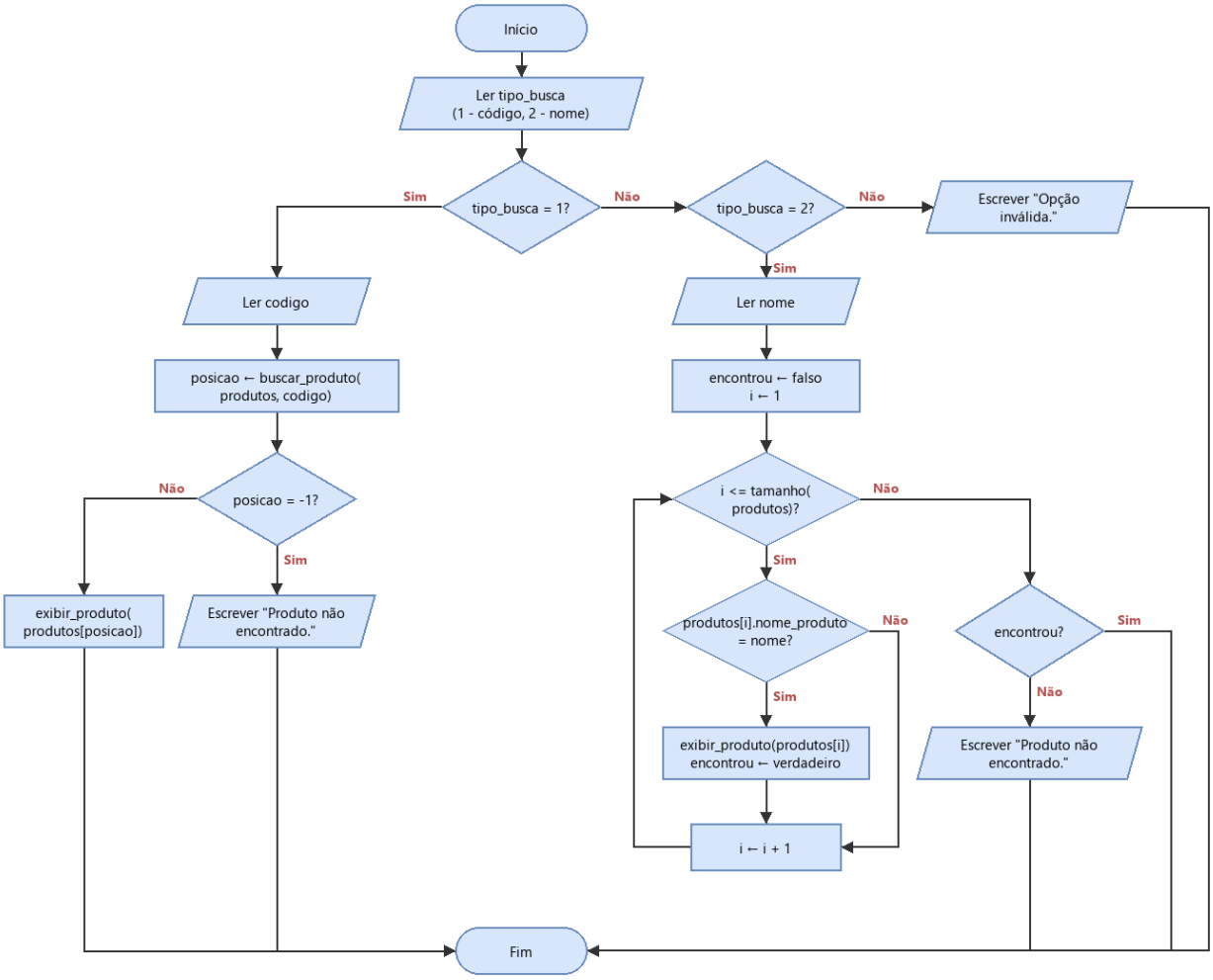
3.3 Saída de produtos

A decisão `quantidade_saida <= quantidade_atual`, em laranja, é a que impede o estoque negativo.



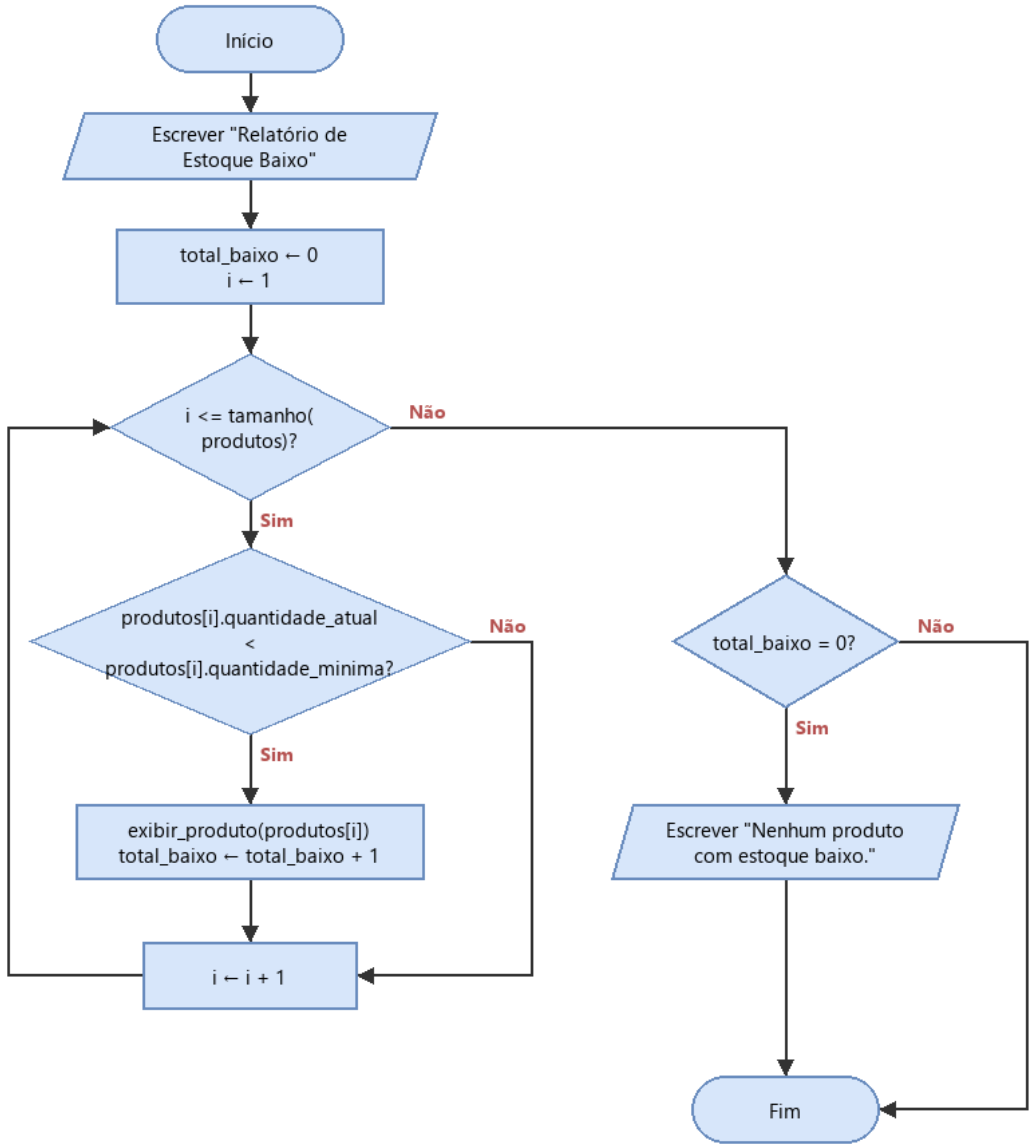
Fluxograma da saída de produtos

3.4 Consulta de produto



Fluxograma da consulta de produto

3.5 Relatório de estoque baixo



Fluxograma do relatório de estoque baixo